

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science & Engineering

ASSESSMENT TOOL 2 – TECHNICAL PRESENTATION

Course Code	CSA0314	Course Title	Data Structures
CO Assessed	CO1	Assessment	Assessment Tool 2: Technical presentation
Weightage	35% of CIA	Total Marks	100
CO4 Statement	<i>Apply suitable data structures and ADTs for efficient problem solving by analyzing time and space complexity. (BL2, BL3)</i>		
Student Name	Kamesh D	Reg. No.	192565003
Section/Batch	Slot C /	Date	03-10-2026

Code of Conduct:

I, kamesh D (192565003), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Kamesh D

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts	
Linkages	25%	Establishes clear, meaningful, and accurate relationships between concepts	Shows correct relationships with minor gaps	Relationships are weak, unclear, or partially incorrect	No meaningful relationships established	
Organization	20%	Concepts are well-structured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure	
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence	
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand	

CO1-AT2-TECHNICAL PROGRAMMING

16). A printer processes multiple print requests in the order they are received. Recommend an appropriate data structure and prepare the presentation explaining your choice



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CSA0314 DATA STRUCTURES

CO1 AT2 TECHNICAL PRESENTATION

GUIDED BY
Dr.Krishanaraj Mony

PRESENTED BY
KAMESH D(192565003)

Printer Request Scheduling: Why FIFO Queue?



Technical analysis of data structures for sequential resource management
in modern operating environments.

Translate the Requirement into Behavior



Arrival Order

Requests must be processed exactly in the order they were submitted. Preservation of chronological sequence is the primary constraint.



Resource Bottleneck

The printer is a single-threaded physical resource. It can only handle one job at a time, necessitating a waiting area for pending work.



Buffering Strategy

When the printer is busy, new incoming requests must be stored without blocking the submitting application.



Fairness

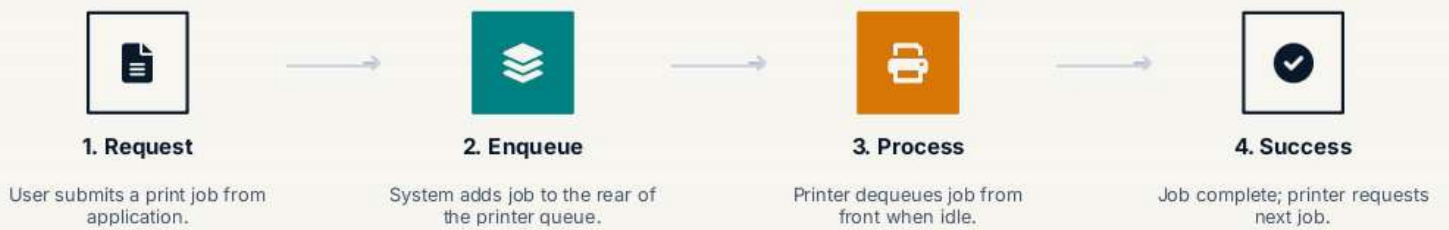
Every request is guaranteed processing; no job should be "starved" or jumped by later arrivals unless explicit priority is specified.

The Queue: First In, First Out (FIFO)



A **Linear Data Structure** that restricts insertion to one end (Rear) and removal to the other (Front). This **FIFO** property perfectly mirrors the "order received" requirement, ensuring no job is bypassed.

From Submission to Printed Output



ASYNCHRONOUS DECOUPLING

The queue acts as a buffer. The application (Producer) finishes its task immediately after `enqueue()`, while the printer (Consumer) processes the jobs at its own physical speed.

Core Operations and Cost

OPERATION	DESCRIPTION	TIME COMPLEXITY
<code>enqueue(job)</code>	Adds a new print request to the rear of the queue.	$O(1)$
<code>dequeue()</code>	Removes and returns the job at the front .	$O(1)$
<code>peek()</code>	Inspects the job at the front without removing it.	$O(1)$
<code>isEmpty()</code>	Returns true if there are no pending requests.	$O(1)$

EFFICIENCY

Constant time $O(1)$ for all primary operations ensures that system overhead remains minimal regardless of the number of waiting jobs.

STORAGE

Linear space $O(n)$ relative to the number of requests. In practice, usually implemented as a circular buffer or a linked list.



Printer Queue — Implementation Sketch

```
class PrinterQueue:
    def __init__(self):
        self.queue = Queue() # FIFO abstraction
self.is_busy = False

def submit_job(self, job):
    self.queue.enqueue(job)
    self.process_next()

def process_next(self):
    if not self.is_busy and not self.queue.isEmpty():
        current_job = self.queue.dequeue()
        self.execute_print(current_job)

def on_complete(self):
    self.is_busy = False
    self.process_next()
```

STATE MANAGEMENT

The system maintains a FIFO queue of pending print jobs and a boolean `is_busy` that reflects the physical printer status.

TRIGGER MECHANISM

`process_next()` is invoked when a new job is submitted and again when a job completes, ensuring continuous processing without polling.

SAFETY CHECK

Before dequeuing, the code verifies the queue is not empty to prevent runtime errors and race conditions.

Worked Example: Arrival Order Preserved

T1: Job A arrives		Printing: A
T2: Job B arrives	B	Printing: A
T3: Job C arrives	C B	Printing: A
T4: A finishes	C	Printing: B

Resulting Output Sequence: A → B → C (Exact matches arrival order)

Production Considerations and Recommendation



Architectural Choice: FIFO Queue

The Queue data structure is the optimal choice for preserving request order, ensuring fairness, and decoupling submission from physical processing.



BOUNDED BUFFERING

Production systems should define a maximum queue size to prevent memory exhaustion during extreme traffic spikes.



CONCURRENCY CONTROL

Implement thread-safe enqueue operations to handle multiple applications submitting jobs simultaneously.



CANCELLATION SUPPORT

While FIFO is standard, allow "search and remove" operations for users to cancel jobs before they reach the front.



PRIORITY OVERRIDES

If high-importance jobs must skip the line, transition from a simple Queue to a Priority Queue (Heaps).